

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Debugging and Optimisation task
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	M.Koushik Reddy	Reg. No.:	192372185
Section / Batch:	CSE-AI	Date:	08-05-2026

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

Code of Conduct

I M.Koushik Reddy / 192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator Faculty Incharge:-

DEBUGGING & OPTIMIZATION TASKS

Detailed Assignment

1. Signed 8-bit Addition (2's Complement Overflow)

Problem Statement

A DSP system produces incorrect results when adding large positive signed 8-bit numbers.

Step-by-Step Debugging Procedure

1. Understand the range of signed 8-bit numbers: -128 to +127.
2. Take A=100 and B=60.
3. Convert both values into 8-bit binary (01100100 and 00111100).
4. Perform binary addition. The result becomes 10100000.
5. Observe that the MSB is 1, therefore the processor interprets the result as a negative number.
6. Convert the result back to decimal using two's complement and obtain -96.
7. Compare the actual answer (160) with the obtained answer (-96).
8. Conclude that the result exceeds the representable range, causing overflow.

Analysis Table

Check	Observation
Operand signs	Positive + Positive
Result sign	Negative
Overflow	Yes

Optimization

Use an overflow detection circuit, overflow flag (Carry into MSB XOR Carry out of MSB), and saturation arithmetic.

Advantages

- Improves accuracy.
- Reduces execution time.
- Increases processor efficiency.
- Produces reliable results.

Conclusion

The debugging process identifies the root cause of the error, while the proposed optimization improves the overall performance and reliability of the system.

2. Ripple Carry Adder and Carry Look-Ahead Adder

Problem Statement

Execution time is high because carry propagates through every full adder.

Step-by-Step Debugging Procedure

1. Study how carry moves from the least significant bit to the most significant bit.
2. Observe that each stage waits for the previous carry.
3. Measure propagation delay.
4. Identify ripple carry as the bottleneck.
5. Replace it with Carry Look-Ahead Adder.
6. CLA calculates carry using Generate ($G=A.B$) and Propagate ($P=A \text{ XOR } B$) signals in parallel.

Analysis Table

Feature	Ripple Carry	Carry Look-Ahead
Carry Generation	Sequential	Parallel
Speed	Slow	Fast
Hardware	Simple	More complex

Optimization

CLA greatly reduces propagation delay and improves ALU performance.

Advantages

- Improves accuracy.
- Reduces execution time.
- Increases processor efficiency.
- Produces reliable results.

Conclusion

The debugging process identifies the root cause of the error, while the proposed optimization improves the overall performance and reliability of the system.

3. Booth's Multiplication Algorithm

Problem Statement

Signed multiplication gives incorrect results for negative numbers.

Step-by-Step Debugging Procedure

1. Initialize A, Q, M and Q-1.
2. Check Q0 and Q-1.
3. Apply Booth rules (00,01,10,11).
4. Perform arithmetic right shift after every iteration.
5. Verify sign extension.
6. Repeat until all iterations are complete.
7. Compare the final answer with expected decimal multiplication.

Analysis Table

Q0Q-1	Operation
00	No operation
01	$A=A+M$
10	$A=A-M$
11	No operation

Optimization

Always use arithmetic right shift and preserve the sign bit.

Advantages

- Improves accuracy.
- Reduces execution time.
- Increases processor efficiency.
- Produces reliable results.

Conclusion

The debugging process identifies the root cause of the error, while the proposed optimization improves the overall performance and reliability of the system.

4. Restoring Division vs Non-Restoring Division

Problem Statement

Repeated restoration increases execution time.

Step-by-Step Debugging Procedure

1. Shift AQ left.
2. Subtract divisor.
3. If negative, restoring division adds the divisor back.
4. Repeat for every bit.
5. Notice that restoration is repeated many times.
6. Replace with non-restoring division.
7. Correct the remainder during the next iteration instead of restoring immediately.

Analysis Table

Algorithm	Main Idea
Restoring	Restore negative remainder
Non-restoring	Correct in next iteration

Optimization

Non-restoring division reduces arithmetic operations and execution time.

Advantages

- Improves accuracy.
- Reduces execution time.
- Increases processor efficiency.
- Produces reliable results.

Conclusion

The debugging process identifies the root cause of the error, while the proposed optimization improves the overall performance and reliability of the system.

5. IEEE 754 Floating Point

Problem Statement

Floating-point addition loses precision when operands have widely different magnitudes.

Step-by-Step Debugging Procedure

1. Represent both values in IEEE-754 format.
2. Compare exponents.
3. Shift the smaller mantissa to align exponents.
4. Add mantissas.
5. Normalize the result.
6. Round to fit the mantissa.
7. Observe precision loss.
8. Use higher precision when necessary.

Analysis Table

Field	Bits
Sign	1
Exponent	8
Mantissa	23

Optimization

Use double precision (64-bit), Kahan summation and numerically stable algorithms.

Advantages

- Improves accuracy.
- Reduces execution time.
- Increases processor efficiency.
- Produces reliable results.

Conclusion

The debugging process identifies the root cause of the error, while the proposed optimization improves the overall performance and reliability of the system.